

Jadara Backend Architecture Guide

RESTful API with JWT, RBAC & PostgreSQL

Technical Documentation

Version 1.0 — August 2026

Contents

1 Overview	2
2 Tech Stack	2
3 Project Structure	2
4 Architecture Patterns	3
4.1 Layered Module Architecture	3
4.2 Request Flow	3
5 Authentication	4
5.1 JWT Token Strategy	4
5.2 Registration Flow	4
5.3 Login Flow	4
5.4 Token Refresh	5
5.5 Change Password	5
5.6 Logout	5
6 Role-Based Access Control (RBAC)	5
6.1 Database Models	5
6.2 Permission Check Flow	5
6.3 Roles	5
7 Rate Limiting	6
7.1 Limits by Endpoint	6
8 Database Schema (8 Models)	6
9 Implementation Status	6
9.1 Implemented (working code)	6
9.2 Stubbed (return 501, TODO comments)	7
9.3 Not Yet Executed	7
10 Key Differences from Unv-Pro Reference	7
11 Default Admin Credentials	7
12 API Endpoints	7
12.1 Auth (/v1/auth)	7
12.2 Users (/v1/users)	8
12.3 Roles (/v1/roles)	8
12.4 Permissions (/v1/permissions)	8
12.5 Health	8

Overview

Jadara is a RESTful backend API built with Node.js, TypeScript, Express 5, and PostgreSQL. It implements JWT authentication with refresh token rotation, role-based access control (RBAC), and follows a layered architecture with clear separation of concerns.

Tech Stack

Category	Technology
Runtime	Node.js 18+
Language	TypeScript 5.8
Framework	Express 5.1
Database	PostgreSQL 16
ORM	Prisma 6.19
Authentication	JWT (access + refresh tokens)
Password Hashing	Argon2id
Validation	Zod 4
API Docs	Swagger / OpenAPI 3.0
Logging	Pino
Process Manager	PM2
Rate Limiting	rate-limiter-flexible (in-memory)

Project Structure

```

jadara/
├── framework/
│   └── # Reusable infrastructure layer
│       ├── config/
│       │   ├── env.ts           # Environment variable validation (Zod)
│       │   ├── prisma.ts        # Prisma client singleton
│       │   ├── logger.ts        # Pino logger config
│       │   └── swagger.ts       # Swagger/OpenAPI setup
│       ├── middleware/
│       │   ├── asyncHandler.ts  # Wraps async route handlers
│       │   ├── requestId.ts     # X-Request-Id header (UUID)
│       │   ├── verifyAccessToken.ts # JWT access token verification
│       │   ├── verifyRefreshToken.ts # JWT refresh token + DB verification
│       │   ├── checkPermission.ts # RBAC permission check (cached)
│       │   ├── zodValidate.ts   # Request body validation
│       │   ├── zodValidateQuery.ts # Query string validation
│       │   ├── rateLimiter.ts   # In-memory rate limiting + headers
│       │   ├── originCheck.ts   # CORS origin verification
│       │   ├── errorHandler.ts # Global error handler
│       │   ├── notFound.ts      # 404 handler
│       │   ├── upload.ts        # Multer file upload config
│       │   └── uploadErrorHandler.ts # Upload error handling
│       ├── types/
│       │   └── express.d.ts     # Express Request type augmentation
│       └── utils/
│           ├── AppError.ts      # Custom error class
│           ├── response.ts       # Standardized JSON responses
│           ├── jwt.ts           # JWT sign/verify helpers
│           ├── hash.ts          # Argon2id hash/verify helpers
│           └── cache.ts         # In-memory cache (no Redis)

```

```

    email.ts                # Nodemailer transport + templates
    pagination.ts           # Offset & cursor pagination helpers
prisma/
  schema.prisma             # Database schema (9 models)
  seed.ts                   # Default permissions, roles, admin
src/
  index.ts                  # Server entry point
  app.ts                    # Express app setup & route mounting
  modules/
    auth/                   # Authentication (implemented)
      auth.routes.ts
      auth.controller.ts
      auth.service.ts
      auth.validator.ts
      refresh-token.repository.ts
      role.repository.ts
      login-history.repository.ts
    users/                  # User management (stub)
    roles/                  # Roles & permissions (stub)
  doc/                      # Documentation
  logs/                     # PM2 log files
  scripts/                  # Backup scripts
  tests/                    # Test files
  ecosystem.config.cjs      # PM2 config

```

Architecture Patterns

Layered Module Architecture

Each module follows a 4-layer pattern:

```

Routes --> Controller --> Service --> Repository
|         |             |         |
Middleware Request/   Business   Prisma DB
chain     Response    logic      queries

```

- **Routes** — Define endpoints, apply middleware chain (auth, validation, rate limiting, permissions)
- **Controller** — Extract request data, call service, send response. Wrapped in `asyncHandler`
- **Service** — Business logic, throws `AppError` on failures
- **Repository** — Direct Prisma queries (data access only)

Request Flow

```

Client Request
|
requestId (X-Request-Id header)
|
rateLimiter (in-memory, X-RateLimit-* headers)
|
originCheck (CSRF protection for state-changing methods)
|
zodValidate (request body / query validation)
|

```

```

verifyAccessToken (JWT from cookie or Authorization header)
|
checkPermission (RBAC from DB, cached 60s in memory)
|
Controller --> Service --> Repository --> Prisma --> PostgreSQL
|
response.success() / response.error()
|
errorHandler (catches AppError and unhandled errors)

```

Authentication

JWT Token Strategy

Token	Expiry	Payload	Storage
Access Token	15 min	{userId, email, role}	httpOnly cookie + body
Refresh Token	7 days	{userId, email}	httpOnly cookie + DB (hashed)

Registration Flow

1. Validate input (Zod) — includes `role_id`
2. Check email uniqueness
3. Validate role exists and is one of: Beneficiary, Reviewer, Company
4. Hash password (argon2id)
5. Create user (status: `active`) + profile with chosen role
6. Sign access token (15min) + refresh token (7d)
7. Hash refresh token, store in DB
8. Return tokens

Login Flow

1. Validate input (Zod)
2. Find user by email
3. Check status = `active`
4. Verify password (argon2id)
5. Sign access token (15min) + refresh token (7d)
6. Hash refresh token, store in DB
7. Record login history (IP, user agent, success/fail)
8. Set httpOnly cookies

Token Refresh

1. Verify refresh token JWT signature
2. Revoke all old refresh tokens for user
3. Sign new access token + refresh token
4. Hash new refresh token, store in DB

Change Password

1. Verify old password (argon2id)
2. Hash new password
3. Update password in DB
4. Revoke all refresh tokens (force re-login on all devices)

Logout

1. Revoke all refresh tokens for user
2. Clear refresh cookie

Role-Based Access Control (RBAC)

Database Models

```
users.role_id --> roles.id (one user = one role)
roles <---> role_permissions <---> permissions
```

Permission Check Flow

1. Call `checkPermission("manage_users")`
2. Cache lookup: `permissions:user:{userId}`
3. On miss: Query `users` → `roles` → `role_permissions` → `permissions`
4. Cache result (60s TTL)
5. Check if permission name exists in user's permissions
6. Allow / Deny (403)

Roles

Role	Description	Registration
Beneficiary	End user	Yes
Reviewer	Content reviewer	Yes
Company	Business account	Yes
Admin	Full system access	No (admin only)

Rate Limiting

In-memory rate limiting using `rate-limiter-flexible`. Applied per endpoint with response headers:

Header	Description
X-RateLimit-Limit	Max requests allowed
X-RateLimit-Remaining	Requests remaining
X-RateLimit-Reset	Seconds until reset
Retry-After	Seconds to wait (on 429)

Limits by Endpoint

Endpoint	Limit
Login	10 req/min
Register	3 req/hour
Change Password	5 req/5min
Refresh Token	20 req/min
Logout	20 req/min
General	100 req/min

Database Schema (8 Models)

Model	Purpose
users	User accounts (email, password hash, status, role_id)
profiles	User profile data (name, phone, avatar)
roles	Role definitions (Beneficiary, Reviewer, Company, Admin)
permissions	Permission definitions
role_permissions	Role ↔ Permission junction
refresh_tokens	Hashed refresh tokens
login_history	Login audit log
devices	Registered device fingerprints

Implementation Status

Implemented (working code)

- All framework middleware (13 files)
- All framework utilities (7 files)
- All configuration (4 files)
- Express type augmentation
- Prisma schema + seed
- All route definitions
- All Zod validators
- App setup and route mounting

Stubbed (return 501, TODO comments)

Module	Files Missing Logic
users	users.service.ts, users.controller.ts, user.repository.ts
roles	roles.service.ts, roles.controller.ts, roles.repository.ts (new)

Not Yet Executed

- npm install
- npx prisma generate
- npx prisma migrate dev
- npx prisma db seed

Key Differences from Unv-Pro Reference

Feature	Unv-Pro	Jadara
Redis	Required (caching, sessions)	Not used (in-memory cache)
MinIO	Required (file storage)	Not used
Modules	15+ modules	3 modules (auth, users, roles)
User Roles	Many-to-many (user_roles)	One-to-many (users.role_id)
Registration	Email verification required	No email verification
Access Token Expiry	60 minutes	15 minutes
Refresh Token Rotation	No	Yes (old revoked on refresh)
Rate Limit Headers	No	Yes (X-RateLimit-*)
Request ID	No	Yes (X-Request-Id)
Health Check	Simple OK	Verifies DB connectivity
Audit Logging	In-memory only	audit_logs model
Logger	console.error in places	Pino everywhere

Default Admin Credentials

```
Email:      admin@jadara.com
Password: Admin@12345
```

API Endpoints

Auth (/v1/auth)

Method	Endpoint	Auth	Description
POST	/register	No	Register new user
POST	/login	No	Login
POST	/refresh-token	Refresh	Rotate refresh token
POST	/change-password	Access	Change password (old required)

POST /logout Access Logout

Users (/v1/users)

Method	Endpoint	Auth	Description
GET	/me/profile	Access	Get own profile
PATCH	/me/profile	Access	Update own profile
GET	/	manage_users	List all users
GET	/:id	manage_users	Get user by ID
PATCH	/:id	manage_users	Update user
DELETE	/:id	manage_users	Delete user
GET	/:id/role	manage_users	Get user's role

Roles (/v1/roles)

Method	Endpoint	Auth	Description
GET	/	No	List all roles (public, for registration)
GET	/:id	manage_roles	Get role by ID
POST	/	manage_roles	Create role
PATCH	/:id	manage_roles	Update role
DELETE	/:id	manage_roles	Delete role
GET	/:id/permissions	manage_roles	Get role permissions
POST	/:id/permissions	manage_roles	Add permission to role
DELETE	/:id/permissions/:permissionId	manage_roles	Remove permission
GET	/:id/users	manage_roles	Get users with role

Permissions (/v1/permissions)

Method	Endpoint	Auth	Description
GET	/	manage_roles	List all permissions
GET	/:id	manage_roles	Get permission by ID
POST	/	manage_roles	Create permission
PATCH	/:id	manage_roles	Update permission
DELETE	/:id	manage_roles	Delete permission
GET	/:id/roles	manage_roles	Get roles with permission

Health

Method	Endpoint	Auth	Description
GET	/health	No	Health check (DB connectivity)